

Práctica 9: Deep Learning

Tratamiento de Señales Visuales, GCID

Domínguez Hernández, Iván

Miranda López, Lucas

Curso 2025–2026

Introducción y objetivo de la práctica

El objetivo de esta práctica es aplicar redes neuronales convolucionales (CNN) para clasificar imágenes de ultrasonido mamario del dataset BUSI (*Breast Ultrasound Images*) [1], siguiendo el flujo del tutorial de MATLAB *View Network Behavior Using t-SNE* [5, 8]. El dataset contiene 780 imágenes distribuidas en tres clases: **benign** (487 imágenes), **malignant** (210) y **normal** (133), junto con sus correspondientes máscaras de segmentación.

El trabajo se divide en dos ejercicios. El primero consiste en adaptar el tutorial para clasificar imágenes con una red preentrenada mediante *transfer learning* [6] y compararla contra clasificadores de referencia triviales. El segundo analiza el impacto de distintos hiperparámetros durante el entrenamiento.

1. Ejercicio 1: Clasificación con red preentrenada

1.1. Preparación de los datos

El dataset se descarga directamente desde los servidores de MathWorks con la función:

```
matlab.internal.examples.downloadSupportFile.
```

Al cargarlo con `imageDatastore`, el directorio contiene tanto imágenes originales como imágenes de máscara (con sufijo `_mask`), por lo que se filtran estas últimas antes de continuar:

```
imds = subset(imds, find(~contains(imds.Files, "mask")));
```

Tras el filtrado se dispone de 780 imágenes. Se dividen en un 65 % para entrenamiento (507 imágenes) y un 35 % para validación (273 imágenes) mediante `splitEachLabel`, que garantiza que la proporción de clases se mantiene en ambos subconjuntos.

Dado que las imágenes de ultrasonido son en escala de grises y GoogLeNet espera entradas RGB de 224×224 píxeles, se usa `augmentedImageDatastore` con la opción siguiente:

```
ColorPreprocessing='gray2rgb'
```

para duplicar el canal de gris en los tres canales de color. Para el conjunto de entrenamiento se aplica además *data augmentation* [7] con volteos horizontales y verticales aleatorios y escalado aleatorio entre 0,8 y 1,2, con el objetivo de aumentar la variabilidad artificial de los datos y reducir el sobreajuste.

1.2. Red neuronal utilizada

Se emplea GoogLeNet, una red de clasificación preentrenada sobre ImageNet (más de un millón de imágenes de 1000 categorías). Mediante *transfer learning*, se sustituye la capa de clasificación final por una nueva capa adaptada a las 3 clases del dataset BUSI, y se reentrenan los pesos con los datos de ultrasonido:

```
net = imagePretrainedNetwork("googlenet", NumClasses=numClasses);  
net = trainnet(augImdsTrain, net, "crossentropy", opts);
```

Esta estrategia tiene sentido porque las capas iniciales de la red ya han aprendido a detectar características visuales genéricas (bordes, texturas, gradientes), que son útiles independientemente del dominio. Solo las últimas capas, responsables de la clasificación específica, necesitan adaptarse al nuevo problema. El optimizador usado es Adam [3] con tasa de aprendizaje 10^{-4} , 50 épocas y *mini-batch* de 32.

1.3. Configuración del entrenamiento y curva de aprendizaje

El entrenamiento se configura con el optimizador Adam, tasa de aprendizaje inicial 10^{-4} , 50 épocas y *mini-batch* de 32 imágenes. La entropía cruzada (*cross-entropy*) se usa como función de pérdida, que es la elección estándar para clasificación multiclase. El tamaño de *batch* de 32 es un compromiso habitual: suficientemente grande para que las estimaciones del gradiente sean estables, y suficientemente pequeño para que quepan varias actualizaciones por época.

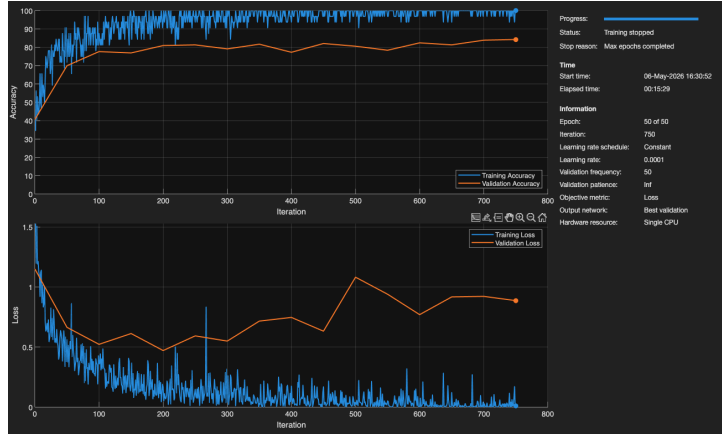


Figura 1: Curva de entrenamiento – pérdida y precisión en entrenamiento y validación a lo largo de las 50 épocas.

Como se observa en la Figura 1, a partir de aproximadamente la época 20 la pérdida en validación deja de mejorar y comienza a aumentar ligeramente, mientras que la pérdida de entrenamiento sigue cayendo. Esto es un síntoma claro de sobreajuste. En este escenario sería recomendable aplicar *early stopping*, deteniendo el entrenamiento cuando la métrica de validación deja de mejorar durante un número determinado de épocas. Cabe señalar que el fenómeno de *double descent* [2], donde la pérdida de test vuelve a bajar tras empeorar al aumentar la capacidad del modelo o el número de épocas, no es esperable aquí: se trata de un efecto que aparece típicamente en modelos de gran capacidad entrenados durante muchas épocas sobre datasets grandes, condiciones que no se dan en este experimento con 507 muestras y 50 épocas.

1.4. Resultados y matriz de confusión

Una vez entrenada la red, se clasifican las imágenes de validación con `minibatchpredict` y se convierten las puntuaciones a etiquetas con `scores2label`:

```
scores = minibatchpredict(net, augImdsVal);
YPred = scores2label(scores, classNames);
```

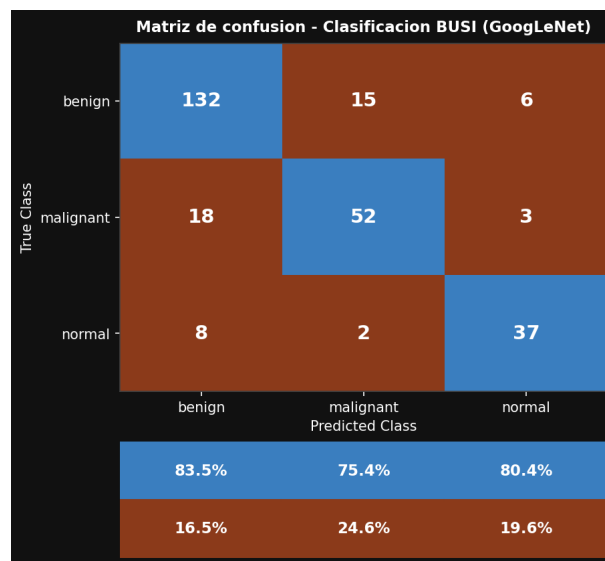


Figura 2: Matriz de confusión – GoogLeNet con *transfer learning* sobre BUSI.

La matriz de confusión (Figura 2) muestra que la red consigue clasificar correctamente la mayoría de imágenes de la clase **benign**, que es la más representada. Las clases **malignant** y **normal** presentan más confusiones, lo cual es esperable dado el desbalance del dataset y la similitud visual entre imágenes de ultrasonido de distintas categorías. El aspecto ecográfico de tumores benignos y malignos puede ser muy parecido incluso para radiólogos experimentados, por lo que un cierto nivel de error de la red es comprensible.

1.5. Comparativa con clasificadores de referencia

Para contextualizar el rendimiento de la red, se compara con dos estrategias triviales:

Clasificador aleatorio: asigna una clase al azar en cada predicción. Con tres clases equiprobables, la precisión esperada es $\approx 33\%$. En la práctica, dado el desbalance del dataset, la precisión real varía ligeramente según la distribución de la clase predicha.

Clasificador constante: predice siempre la clase más frecuente del conjunto de entrenamiento, que es **benign**. Su precisión en validación coincide con la proporción de imágenes benignas, aproximadamente el 62%. Aunque numéricamente parece un resultado aceptable, este clasificador es completamente inútil en la práctica: nunca detecta casos malignos ni normales.

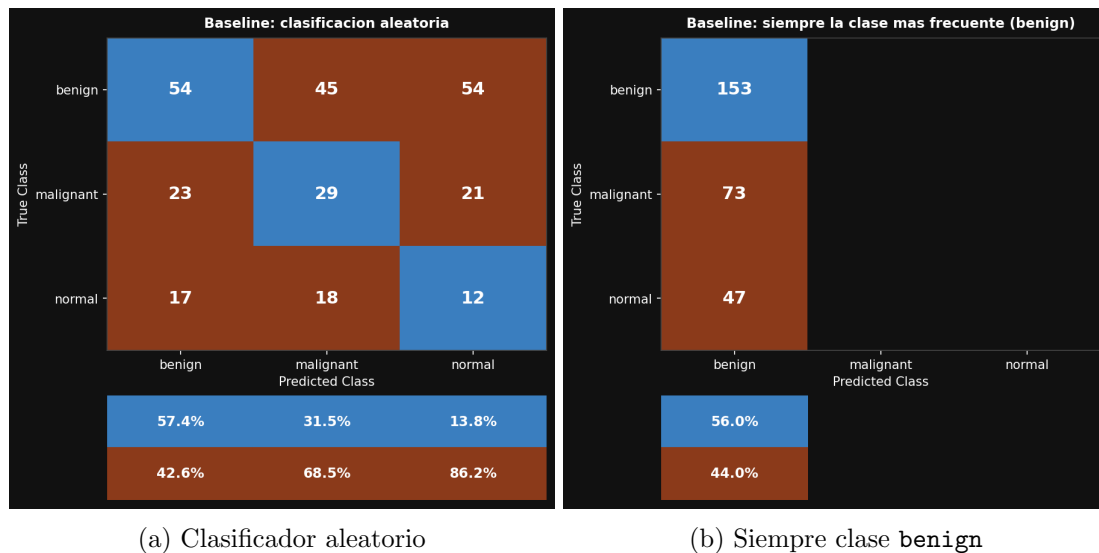


Figura 3: Matrices de confusión de los clasificadores de referencia.

La comparativa pone de manifiesto que la red entrenada supera claramente al clasificador aleatorio en todas las clases. Respecto al clasificador constante, aunque este tiene mayor precisión global por el desbalance, la red identifica correctamente ejemplos de las tres clases, lo que la hace mucho más útil en un contexto clínico donde no detectar un tumor maligno tiene consecuencias graves.

2. Ejercicio 2: Comparativa del proceso de entrenamiento

En este ejercicio se entrena GoogLeNet sobre solo el 10% de los datos (aproximadamente 78 imágenes para entrenamiento y el resto para validación) para agilizar los experimentos. Se evalúa el efecto de tres factores sobre el rendimiento.

2.1. Con y sin *data augmentation*

El *data augmentation* introduce variaciones artificiales en las imágenes de entrenamiento (volteos y escalados aleatorios), de modo que la red ve mayor diversidad en cada época aunque el conjunto de datos sea pequeño. Con un 10% de los datos el sobreajuste es especialmente acusado, por lo que se espera que el *augmentation* mejore la generalización.

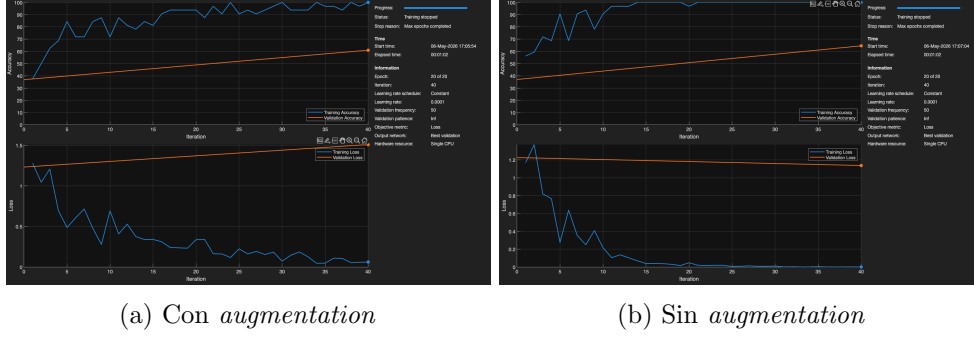


Figura 4: Curvas de aprendizaje con y sin *data augmentation*.

Las curvas muestran que con *augmentation* la pérdida de validación desciende de forma más suave y estable. Sin *augmentation*, la red tiende a memorizar los pocos ejemplos disponibles: la pérdida de entrenamiento cae rápido pero la de validación se estanca antes o incluso sube, señal clara de sobreajuste.

2.2. Variación del número de épocas

Se entrena la red con 5, 20 y 50 épocas manteniendo el resto de hiperparámetros fijos (*MiniBatchSize*=32, con *augmentation*).

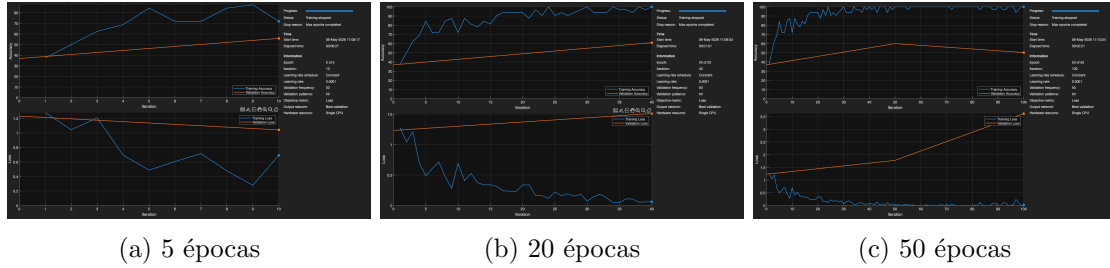
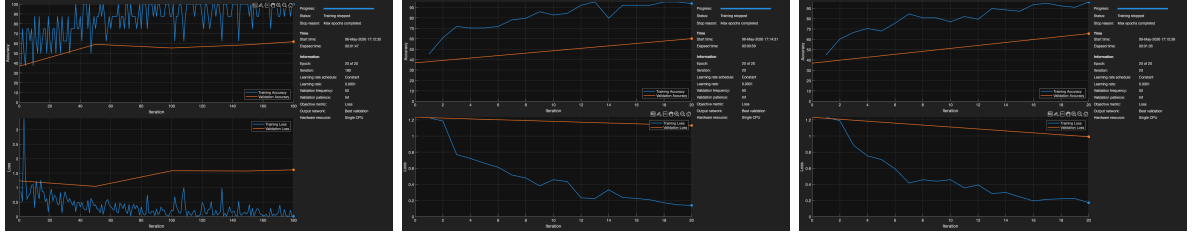


Figura 5: Curvas de aprendizaje para distintos números de épocas.

Con 5 épocas la red aún no ha convergido: la pérdida de validación sigue cayendo y la precisión es baja. Con 20 épocas se obtiene un equilibrio razonable entre entrenamiento y generalización. Con 50 épocas, dado el escaso número de muestras, la red empieza a sobreajustarse: la pérdida de entrenamiento sigue bajando pero la de validación se estanca o sube.

2.3. Variación del *MiniBatchSize*

Se comparan *mini-batch* de 8, 64 y 128 muestras con 20 épocas y *augmentation*.



(a) $MiniBatchSize=8$

(b) $MiniBatchSize=64$

(c) $MiniBatchSize=128$

Figura 6: Curvas de aprendizaje para distintos valores de $MiniBatchSize$.

Con *batch* pequeño (8) las curvas son más ruidosas porque cada actualización de gradiente se calcula sobre muy pocas muestras. Con *batch* grande (128), especialmente con solo el 10 % de los datos, el número de actualizaciones por época es muy reducido y el aprendizaje puede ser más lento. El valor intermedio (64) tiende a ofrecer curvas más estables, aunque con un dataset tan pequeño las diferencias dependen fuertemente de la aleatoriedad del proceso.

3. Dificultades encontradas

La principal dificultad fue de naturaleza técnica con MATLAB. La función `downloadTrainedNetwork` [4], usada en el tutorial de segmentación, no está disponible como función estándar del toolbox y fue necesario reemplazarla manualmente con `websave` y `unzip`. Además, el dataset BUSI incluye imágenes de máscara junto a las originales con el mismo nombre más el sufijo `_mask`, lo que requiere un filtrado explícito al construir el `imageDatastore` para evitar que se procesen como imágenes de entrenamiento.

Otro problema significativo fue la incompatibilidad entre las imágenes en escala de grises del dataset BUSI y la entrada esperada por GoogLeNet (RGB), resuelta con la opción

```
ColorPreprocessing='gray2rgb'.
```

Por último, MATLAB no soporta aceleración por GPU en macOS mediante CUDA, lo que aumentó considerablemente los tiempos de entrenamiento al ejecutar todo en CPU.

4. Conclusiones

Esta práctica ha permitido aplicar *transfer learning* para clasificación de imágenes médicas, siguiendo el flujo del tutorial de MATLAB adaptado al dataset BUSI. Los resultados del Ejercicio 1 muestran que una red preentrenada con *transfer learning* supera claramente a clasificadores triviales, aunque el desbalance de clases (62 % de imágenes benignas) hace que la precisión global no sea el mejor indicador de rendimiento: en un contexto clínico, los falsos negativos en la clase **malignant** son mucho más costosos que los falsos positivos.

Del Ejercicio 2 se extrae que el *data augmentation* es especialmente beneficioso cuando los datos son escasos, que un número excesivo de épocas conduce a sobreajuste con dataset pequeño, y que el $MiniBatchSize$ óptimo depende del tamaño del conjunto de entrenamiento disponible. En general, la limitación principal de estos experimentos es precisamente la reducción al 10 % de los datos, que hace que todos los modelos entrenados tengan un rendimiento inferior al que se obtendría con el dataset completo.

Referencias

- [1] Walid Al-Dhabyani, Mohammed Gomaa, Hussien Khaled, and Aly Fahmy. Dataset of breast ultrasound images. *Data in Brief*, 28:104863, 2020.
- [2] Mikhail Belkin, Daniel Hsu, Siyuan Ma, and Soumik Mandal. Reconciling modern machine-learning practice and the classical bias–variance trade-off. *Proceedings of the National Academy of Sciences*, 116(32):15849–15854, 2019.
- [3] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.
- [4] MathWorks. Breast tumor segmentation from ultrasound using deep learning. <https://www.mathworks.com/help/medical-imaging/ug/Breast-Tumor-Segmentation-from-Ultrasound-Using-Deep-Learning.html>, 2024. Accedido: mayo 2026.
- [5] MathWorks. View network behavior using t-SNE. <https://www.mathworks.com/help/deeplearning/ug/view-network-behavior-using-tsne.html>, 2024. Accedido: mayo 2026.
- [6] Sinno Jialin Pan and Qiang Yang. A survey on transfer learning. *IEEE Transactions on Knowledge and Data Engineering*, 22(10):1345–1359, 2010.
- [7] Connor Shorten and Taghi M. Khoshgoftaar. A survey on image data augmentation for deep learning. *Journal of Big Data*, 6(1):60, 2019.
- [8] Laurens van der Maaten and Geoffrey Hinton. Visualizing data using t-SNE. *Journal of Machine Learning Research*, 9:2579–2605, 2008.